C1.4 Transaction overview

This section introduces the different transaction types. It provides information about when the transactions are used and the required behavior of the various system components. The section describes:

- *Non-snooping transactions*
- *Coherent transactions*
- *Memory update transactions* on page C1-143
- *Cache maintenance transactions* on page C1-144
- *Snoop transactions* on page C1-144
- *Barrier transactions* on page C1-144
- *Distributed virtual memory transactions* on page C1-145.

C1.4.1 Non-snooping transactions

Non-snooping transactions are used to access memory locations that are not in the caches of other master components. These transactions do not cause snoop transactions to be performed and are used for the following transaction types:

- Non-shareable
- Device.

Two forms of non-snooping transaction are provided, ReadNoSnoop and WriteNoSnoop.

Note

Within the context of coherency, ReadNoSnoop and WriteNoSnoop transactions are also referred to as Read and Write transactions. The extended form of the name can be used to differentiate between this transaction type and the more generic set of all read or write transactions.

C1.4.2 Coherent transactions

In general, coherent transactions are used to access shareable address locations, which might be held in the coherent caches of other components.

Transactions for performing load operations from shareable locations

When a master is required to perform a load operation from a location in a shareable area of memory, the following snoop transactions can be used, all of which permit the current holders of the cache line to retain their copy:

ReadClean The ReadClean transaction indicates that the master component requesting the read can only accept a cache line that is clean, that is, it cannot accept responsibility for a dirty line that it must subsequently write back to memory. Typically, the ReadClean transaction is used by master components that do not have the ability to accept a dirty cache line, or have a write-through cache.

ReadNotSharedDirty The ReadNotSharedDirty transaction indicates that the master requesting the read can accept a line that is in any state except SharedDirty. This means that the line can be passed as clean (either unique or shared) or the line can be passed as unique and dirty.

ReadShared The ReadShared transaction indicates that the master component requesting the read can accept a cache line in any state.

For each of these transactions, it is acceptable for a cache that is being snooped to pass a cache line as dirty, even if it cannot be accepted by the master component that is requesting the cache line. In this situation, the interconnect is responsible for writing back the dirty line to main memory.

If a cache that receives one of these snoop transactions has a copy of the data, this specification recommends that it provides the data to complete the snoop transaction. The interconnect must pass the data back to the initiating master component.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

If the cache that provides the data originally held the line in a Unique state then to retain the copy, it must move the cache line to a Shared state after the operation.

Transactions for performing store operations to shareable locations

When a master is required to perform a store to a location in a shareable area of memory, the following transactions can be used, all of which ensure that there are no other copies of the location when the store operation occurs:

ReadUnique A master component uses the ReadUnique transaction when performing a partial cache line store, that is, when storing only some of the bytes of the cache line, in cases where it does not already have a copy of the cache line. The ReadUnique transaction obtains a copy of the data and ensures that no other copies exist.

CleanUnique A master component uses use a CleanUnique transaction when performing a partial cache line store, in cases where it already has a copy of the cache line. The CleanUnique transaction removes all other copies of the cache line, but if it finds a cache that holds the line in a Dirty state then the transaction ensures that the dirty cache line is written to main memory.

MakeUnique A master component uses the MakeUnique transaction when performing a full cache line store. The MakeUnique transaction invalidates all other copies of the cache line.

Transactions for accessing shareable locations when no cached copy is required

When a master is required to access a shareable memory location but the issuing master is not going to keep a cached copy of the address, either because it does not want to allocate that cache line or because it does not have a cache, the following transactions can be used:

ReadOnce A master component uses the ReadOnce transaction to obtain a snapshot of data that it does not require to copy to its cache. For the ReadOnce transaction, if the cache that provides the data holds the cache line in a Unique state, there is no requirement to change the cache line to a Shared state after the ReadOnce transaction.

WriteUnique

A WriteUnique transaction can be used to remove all copies of a cache line before issuing a write transaction. The WriteUnique transaction can be used when writing a full or partial cache line, and ensures dirty data is written to memory before performing the write transaction.

WriteLineUnique

A WriteLineUnique transaction can be used to remove all copies of a cache line before issuing a write transaction. The WriteLineUnique transaction must only be used when writing a full cache line, where all bytes within the cache line are written by the transaction.

Note

Unlike other transactions to shareable memory, ReadOnce and WriteUnique transactions issued by a master component are not required to be a full cache line size. However, WriteLineUnique transactions are required to be a full cache line size.

C1.4.3 Memory update transactions

The following transactions are used to update main memory:

WriteBack A master component cache uses a WriteBack transaction to write back a dirty line to main memory to free a cache line that is to be used for a different address. The master component does not retain a copy of the cache line.

WriteClean A master component cache uses a WriteClean transaction to write a dirty line to main memory, while permitting that master component to retain a copy of the cache line.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Evict A master component uses an Evict transaction to indicate the address of a cache line that is evicted from its local cache when no main memory update is required. The transaction enables cache lines in a particular component to be tracked, and can be used for applications such as constructing snoop filters. No data transfer is associated with Evict transactions.

Note

The WriteBack, WriteClean, and Evict transactions do not result in snoop transactions to other caches. Other caches are not required to know whether the cache line has been written to main memory. WriteBack, WriteClean, and Evict transactions are not serialized in the same way as other snoop transactions.

C1.4.4 Cache maintenance transactions

Master components use broadcast cache maintenance transactions to access and maintain the caches of other master components in a system. In particular, cache maintenance transactions enable master components to view the effect of load and store operations on system caches that cannot otherwise be accessed. This process is typically referred to as Software Cache Maintenance. Broadcast cache maintenance operations can also propagate to downstream caches, permitting all caches in a system to be maintained.

Note

A master component that initiates a cache maintenance transaction is also responsible for performing the appropriate operation on its own local cache.

The following transactions are used to maintain caches:

CleanShared A master component uses a CleanShared transaction to perform a clean operation on the caches of other components in the system. If a snooped cache that holds a dirty cache line receives a CleanShared transaction, it must provide that cache line so that the cache line can be written to main memory. The snooped cached can retain its local copy of the cache line.

CleanInvalid A master component uses a CleanInvalid transaction to perform a clean and invalidate operation on the caches of other components in the system. If a snooped cache that holds a clean cache line receives a CleanInvalid transaction, it must remove its local copy of the cache line. If a snooped cache that holds a dirty cache line receives a CleanInvalid transaction, it must provide that cache line so that the cache line can be written to main memory and remove its local copy of the cache line.

MakeInvalid A master component uses a MakeInvalid transaction to perform an invalidate operation on the caches of other components in the system. If a snooped cache receives a MakeInvalid transaction, it must remove its local copy, but it is not required to provide any data, even if the cache line is dirty.

C1.4.5 Snoop transactions

Snoop transactions are transactions that use the snoop address, snoop response, and snoop data channels. Snoop transactions are a subset of coherent transactions and cache maintenance transactions.

C1.4.6 Barrier transactions

The ACE protocol supports barrier transactions that are used to provide guarantees about the ordering and observation of transactions in a system.

The following types of barrier are supported:

- memory barrier
- synchronization barrier.

A master component issues a memory barrier to guarantee that if another master in the appropriate domain can observe any transaction after the barrier it must be able to observe every transaction prior to the barrier.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

A master component issues a synchronization barrier to determine when all transactions issued before the barrier are observable by every master in a particular domain. Some synchronization barriers also have a requirement that all transactions issued before the barrier transaction must have reached the destination slave component before the barrier completes.

See Chapter C8 *Barrier Transactions* for more information.

C1.4.7 Distributed virtual memory transactions

Distributed Virtual Memory (DVM) transactions are used for virtual memory system maintenance, and typically pass messages between components within distributed virtual memory systems.

See Chapter C12 *Distributed Virtual Memory Transactions* for more information.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential